

JWT: JSON Web Tokens

Sp 26 - System Analysis & Design

JWT

- JSON Web Token is an open standard (RFC 7519)
- Defines a compact and **self-contained** way for securely transmitting information between parties as a JSON object.
- Can be verified and **trusted** because it is digitally signed.
- Can be signed using a secret (using HMAC) or a key pair (RSA) or ECDSA.

When to use

- Authorization
 - Most common scenario.
 - Once the user is logged in, every request will include the JWT.
- Information Exchange
 - Defines a compact way to transmit signed JSON claims between parties.
 - You can **verify** the sender because they are **signed**
 - Signature is calculated using *header* and *payload* information.
 - You can also be sure that the information is not tampered with.

Structure

- Consists of three parts:
 - Header
 - Payload
 - Signature
- Looks like: **xxxxx.yyyyy.zzzzz**

Header

- Typically consists of two parts:
 - Type of token (JWT)
 - Signing algorithm (HMAC SHA256 or RSA)
- Later, this JSON is encoded in **Base64Url**.

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Base64

- Base64 is used to encode binary data (images, files, etc) into **text**
- Can be easily transmitted between systems which only handles text (emails, JSON, urls, etc)
- Uses characters: A-Z a-z 0-9 + / and uses = or == as **padding** to make the output a multiple of 4.
- This is **not** encryption. It is **encoding**

```
echo any car | base64
```

```
YW55IGNhcgo=
```

Base64Url

- This is another encoding technique which works like Base64; but
 - Make it safe for **urls** and **filenames**
- Uses A-Z a-z 0-9 - _
- If you use *any car* again and use base64url:
 - YW55IGNhcgo → without any =
 - They are replaced.
 - If there were + or / characters, they would also be replaced.
- Used in JWT's and **web-safe** systems.

Payload

- Second part of the token.
- Here, we have **claims** (information)
- Claims are *statements* about the **user** and **additional data**
- Three types of claims:
 - Registered (common: who, when, exp)
 - Public (role)
 - Private (custom between two parties)

Registered Claims

- Set of predefined claims for common fields.
- Not mandatory, but recommended.
- Provides a set of useful, interoperable claims.
 - iss → issuer
 - exp → expiration time
 - sub → subject
 - aud → audience
- and others.
- All are **three** letters long because JWT is meant to be **compact**

Public claims

- We can define them.
 - They are custom, but shared.
- Since they are *shared*, there should be no name collisions.
- For that, there is a registry.
 - IANA JSON Web Token Registry
- If the API is *shared* across companies, you do not need it.
 - OAuth, OpenID connect etc.

Private claims

- These are **custom** claims created to share information between parties that agree on using them
- These are **not** registered or **public** claims
- This is later encoded into Base64Url and it forms the **second part** of JWT.

```
{  
  "sub": "1234567890",  
  "name": "John Doe",  
  "admin": true  
}
```

- In real life we use **registered claims**
 - who the user is **sub**
 - when it expires **exp**
 - when it was issued **iat**
- We use **private claims** for app-specific needs
 - user roles
 - user permissions
 - company id
 - theme preferences, etc.

Example

```
{  
  "sub": "1a2b3c",  
  "exp": 1714500000,  
  "iat": 1714496400,  
  "role": "admin",  
  "user_id": "1a2b3c",  
  "email": "user@example.com"  
}
```

Important

- In *signed tokens*, this part can be read by anyone, unless it is **encrypted**
- **Do not put** secret information in payload or header elements if it is **not** encrypted.
- Meaning that if the token is **not** encrypted, it can be read by anyone.
 - But cannot be changed.
 - If so, server can understand it.

Signature

- To create this part, we have to take the **encoded header**, **encoded payload**, **a secret** and we sign it with the algorithm specified in the header.
- For example, if you want to use HMAC SHA256 algorithm, the signature will be created as follows.

```
HMACSHA256(  
    base64UrlEncode(header) + "." +  
    base64UrlEncode(payload),  
    secret)
```

- The signature is used to verify the message **was not** changed along the way
- If the tokens are signed with a private key, it can also verify that the sender of the JWT is who it says it is.

- As an output, we have a structure separated by a dot (.) and three different base64url encoded data.
- Check this website out:
 - <https://www.jwt.io/#debugger-io>

JWT Signature Algorithms

HS256

RS256

HMAC SHA256

RSA SHA256

Symmetric

Asymmetric

One shared secret

Public/private key pair

Same key signs/verifies

Private signs, public verifies

Simpler

Better for distributed systems

Typical JWT Processing Flow

- Server receives
 - Authorization: Bearer <access token>
- Server splits:
 - header, payload, signature
- It verifies the signature.
 - If the signature is mathematically valid.
 - If not, **reject**
- Signature verification is not enough.
- Why?
 - An old token will still have a valid signature if it is not tampered with.
 - Can be a token from another service.
 - **So after verification, you need to validate.**

Validation

- Check the expiration.
 - Maybe the time for the token is expired?
- Check the audience
 - Maybe it is for a mobile-api but someone is using it for admin-api ?

API key vs API token

- They are not the same.
 - API key is usually a simple string
 - It identifies us to the API.
 - Usually just proves who we are.
- API token is different.
 - More advanced, can **identify** us and can carry **extra** information.
 - As to what we are allowed to do and for **how long**
 - It can expire and can have permissions attached.
 - This is often a JWT. (Not always)

API token

- It is often a JWT but not always.
- It can be:
 - **Opaque Token**
 - Random looking string, only server knows what it is.
 - e.g. (a8f91c0e7b5c4d7a9e2f1, 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d, etc.)
 - **PASETO (Platform-Agnostic Security Tokens)**
 - Modern alternative to JWT, safer and simpler. Not so popular at the moment but gaining traction especially for security-sensitive applications.
 - **SAML token**
 - XML based token used mainly in enterprise SSO (Single Sign-On).
 - You log into a service (corporate portal, azure, etc) SAML tells another app that this employee is authenticated so they can access other tools (jira, salesforce, etc)
 - **Custom token**
 - Something that we invented.

JWT

- In a key, you don't have user info and it cannot expire.
 - In a JWT, you have information and it has an expiration date.
 - exp is optional.
- A Key looks like a long random string.
 - A JWT is a long encoded string.
- A key can be good for very simple stuff.
 - A JWT is good for secure and scalable apps.

API Key

- If I build a public API, I might choose to use an API key.
- When someone signs up, we create a key for that.
 - We store in the database.
- While doing that, we are going to add options for that user.
- So, when someone sends a request to our API we will read the API key, look it up in the db and check their plans and limits.
- We give permission or block the user based on *that* information.
- How would you design such a table in your database?

```
API Key: abc123
Customer Name: Company A
Plan: Free
Max Requests per Day: 1,000
```

JWT

- A JWT is self-contained.
- It holds all the information inside it.
 - Such as user ID, roles, expiration time.
- exp here is unix timestamp.
 - Shows the number of seconds passed til 1970.

```
{  
  "sub": "user123",  
  "role": "admin",  
  "exp": 1714456400  
}
```

- When a server receives JWT, it **decodes** it first.
- Later, it checks the **signature**
- It checks the **exp** field.
- If the signature is correct **and** expiration is not passed, it is **allowed**.

JWT

- Traditional HTML forms cannot send Authorization headers directly.
- JWTs are commonly sent:
 - via Authorization headers using Javascript
 - or via **cookies**

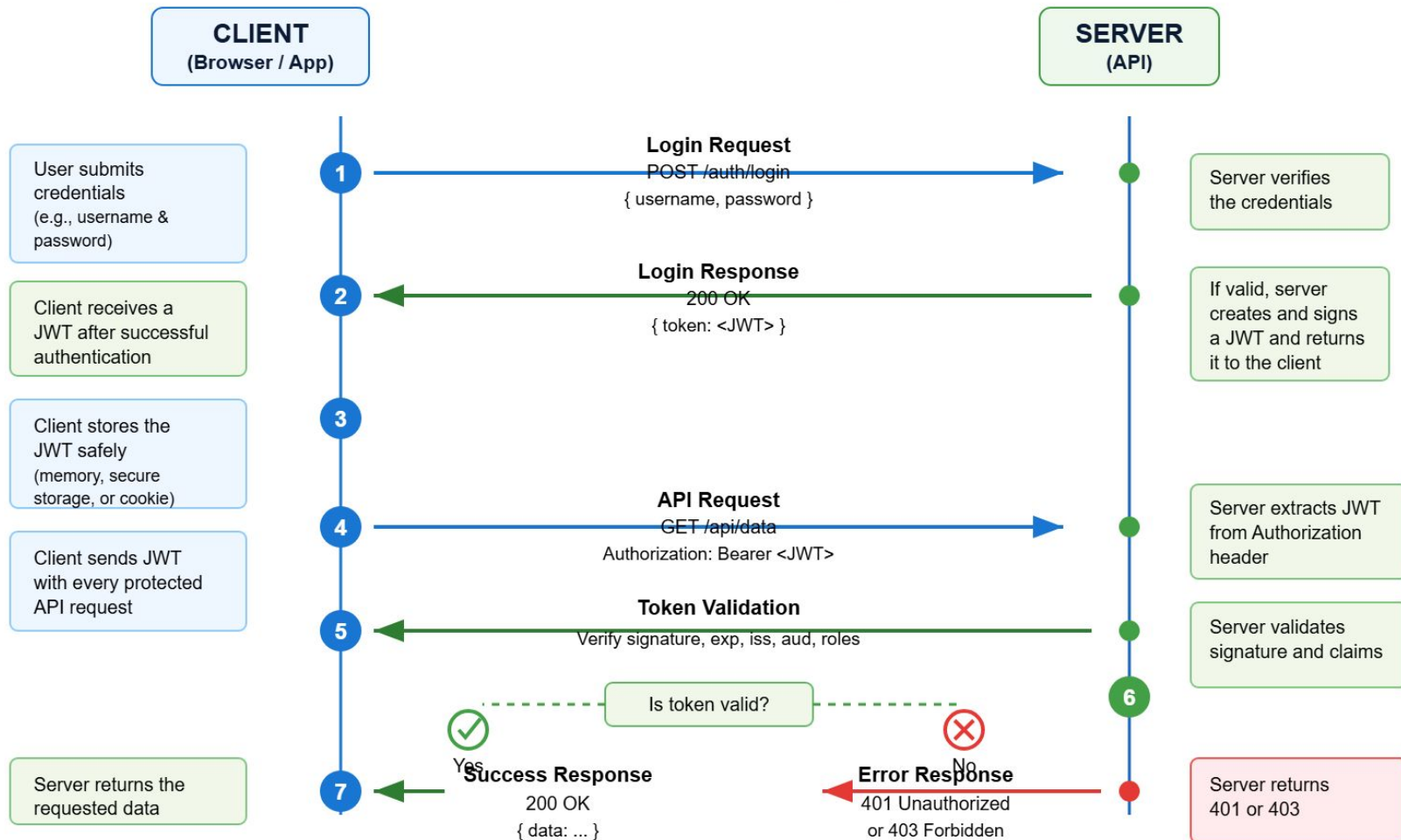
JWT

- JWT is a token format.
- Bearer token is a **way** to send any token.
- So, the ticket will be the JWT (with date and time) but you holding the ticket in your hand and showing is Bearer token method.
- Bearer token is the standard way to send a token to a server.
 - Can be JWT, a string, etc.
- When you choose Bearer in postman, it adds the Bearer to your token (whatever that is)

JWT

- JWT's are not encrypted by default.
- One can choose to encrypt them.
 - Especially if they are storing some secret inside it.
 - But you should not store important data in JWT anyway.
- Signature makes it **tamper proof**, not **private**.

JWT Authentication – Request Flow



Storing JWT

- JWT security is also about **where** you store the token.
- You can store it in several places.
- Most common:
 - localStorage
 - sessionStorage
 - Lives for the lifetime of browser tab/session.
 - JS can still read it.
 - HttpOnly Cookie

localStorage

- Browser-side persistent storage
- Accessible through Javascript
- `localStorage.setItem("jwt", token)`
- Data survives
 - refresh, restart, reopening browser
- Unlike cookies:
 - Browser does NOT automatically send it.
 - JavaScript manually reads/sends it.
- You get the token, you put it in your fetch command, Authorization header.

localStorage

- Easy and simple to use.
 - No backend cookie handling.
 - No CSRF concerns initially.
- User stays logged in after refresh, restart, across tabs, etc.
 - convenient
- SPA tutorials promoted it for years.
- The problem is XSS.
 - Cross-Site Scripting

XSS

- Attacker can inject JavaScript to your website.
- Browser executes it as trusted code.
- If the attacker is able to do it, they can steal the token.
- Now, they can have
 - Authenticated session
 - User identity
 - Permissions
 - etc.

- If JWT is a valid JWT
 - Stealing JWT = stealing login session
- They do **NOT** need:
 - Password
 - MFA
 - email access
- Just the token is enough.

localStorage and XSS

- XSS is still bad even if you don't store the token in localStorage.
- But if there is a XSS and you store the token in localStorage
 - This is a huge risk.
- XSS prevention is hard and dependencies are very large.
- You end up trusting
 - Your code, npm ecosystem, analytics scripts, browser extensions, etc.
- Alternative is storing the token in Cookies.

Cookies

- If the token is stored in Cookies (HttpOnly)
 - That means a JS cannot read it.
 - Because **httpOnly** means you cannot read it with **document.cookie**
- Cookies has built-in security features
 - HttpOnly, Secure, SameSite, etc.
-

Access tokens & Refresh tokens

- How do we keep users logged in securely without making stolen tokens valuable forever?
- **Access token** → short-lived credential used constantly
- **Refresh token** → long-lived credential used rarely to get **new** access tokens.

- Suppose that login gives you a JWT valid for 30 days.
- If attacker steals it
 - They own your account for 30 days.
- So people tried to find a solution to this.
- **Can we make stolen tokens expire quickly without forcing users to log in every 5 minutes?**
 - Refresh tokens to the rescue.

Access token

- Short-lived authentication token
- Usually sent with API requests and contains user identity + permissions
 - The thing we sent as Authorization: Bearer ...
- Lifetime depends on the use case
 - Banking → 5-15 minutes
 - Enterprise → 15-60 minutes
 - Consumer apps → 1-2 hours
- Modern systems prefer shorter lifetimes.

Access token

- Usually contains
 - user id → (sub)
 - roles
 - permissions
 - expiration (exp)
 - issuer (iss)
 - audience (aud)
- Purpose
 - We try to reach an API.
 - Server checks signature, expiration, claims
 - If valid, request accepted.

Refresh token

- The purpose of refresh token is not to access an API.
- Purpose is to obtain **new** access tokens.
 - It is a session renewal credential
- Lifetime
 - Banking → hours / days
 - Enterprise → days / weeks
 - Consumer apps → weeks / months

Flow

1. User logs in and server returns a JSON with **accessToken** and **refreshToken**
2. Client sends the access token in header and it works until expiration.
3. Access token expires. Server returns 401 Unauthorized.
4. Client sends the refresh token (POST /refresh)
5. Server verifies refresh token and if valid, issues a new access token (maybe a new refresh token as well)
6. User never notices and keeps logged in!

Why?

- Short-lived access token limits the theft damage.
- Long-lived refresh token maintains the UX.

Best Practice

- Access token

- Stored in memory only to
 - Minimize persistence
 - Reduce theft window

- Refresh token

- HttpOnly Secure SameSite cookie
 - long-lived
 - highly sensitive
 - should resist JS theft
- Stealing refresh token is much worse than stealing access token.
- Because now the attacker can refresh it any time.

Refresh token rotation

- Often
 - In every refresh, old refresh token is invalidated
 - New refresh token is issued.
 - Helps with theft / replay attacks
- Suppose attacker steals the refresh token
 - If there is no rotation, they can refresh forever.
 - With rotation
 - Both victim and attacker try using the same token
 - Server detects duplicate use
- Does not have to be JWT
 - Can be opaque, random, etc.

AT & RT

- Access tokens are typically JWTs and server verifies the signature and expiration
 - You do not have DB lookup
- Refresh tokens are often stateful and usually stored in DB.
 - Refresh token is bind to user.
 - You have columns to see if revoked and expiration date etc.
- When you logout, you delete/revoke the refresh token.
 - That means no new access tokens can be issued anymore.
 - Access token may still live briefly until expiration.
 - That is why they are **short-lived**

Final

- We login, server returns access token and refresh token.
- Refresh token is stored in cookie.
- Access token is stored in **memory**
 - Just a javascript variable
- Frontend can add this token into any fetch.
- When the page refreshes, access token is lost.
 - But the refresh token still is there.
- App silently refreshes by using (POST /refresh) and receives the new access token.
- This is the common scenario in **SPA**. (Single Page Application)

Final 2

- Can add the access token to the cookie as well.
 - Use it from there.
 - Very common.
- Why not use it always?
 - Introduces CSRF considerations.
 - Because HttpOnly protects against XSS yes.
- Important Cookie Flags
 - Almost always:
 - HttpOnly, Secure, SameSite=Lax or Strict

Architecture	UX	Complexity	XSS Resistance	CSRF Concerns
Access cookie only	Medium	Low	Good	Yes
Access + refresh cookies	Excellent	Medium	Good	Yes
Access memory + refresh cookie	Excellent	Higher	Better	Lower
localStorage JWT	Good	Low	Weak	Lower

How to use?

- **Beginners:**
 - HttpOnly access-token cookie only
- **Intermediate**
 - Access + refresh cookies
- **Advanced**
 - Memory access token + refresh cookie

Common Patterns

- **A: Traditional Session Cookie**
 - Server-side session. - very common and secure
- **B: Access Cookie Only**
 - Simple JWT cookie auth. - very common
- **C: Access + Refresh Cookies**
 - Modern persistent login architecture - very common
- **D: Access in memory + Refresh cookie**
 - Common in SPAs. - More frontend-heavy architecture.

JWT

- It is also still possible to just use the **access token** without **refresh token**
- When the token expires, user logs in again.
 - Much simpler
 - Can be more secure.
- Some systems intentionally force re-auth frequently
 - Banking
 - admin panels
 - military systems
 - etc.
- Worse UX. (Tradeoff)
- Bad for mobile apps.

Problem with cookies

- Browsers send cookies automatically.
- If they see a request going to a certain website
 - They add the relevant cookie immediately
- Imagine that I am adding a post to your website
 - ``
 - Browser requests image URL and cookie automatically is attached.
 - Account is deleted
- For that we need to adjust settings on cookies.
 - SameSite=Lax or SameSite=Strict
 - Lax: Blocks many dangerous cross-site POSTs. Modern default in many browsers.
 - Strict: Very strong, never sends cookie cross-site. Sometimes breaks login flows.
 - None: Allows cross-site cookie sending.

CSRF Tokens

- Classic defense
- Server generates random token
 - `<input type="hidden" name="csrfToken" value="abc123">`
 - Server validates token on submission.
 - Another site cannot know the token value.
- Server can also check the “origin” of the request and reject suspicious origins.
- Modern browsers improved CSRF significantly with
 - SameSite

When NOT to use JWT

- JWT may be unnecessary and over-engineering:
 - Monolithic app
 - Traditional web app
 - Simple session authorization
 - Revocation is important

Common JWT Mistakes

- Storing secrets in payload
- Long expiration date
- No signature validation
- localStorage theft risks
- using JWT when sessions are simpler
- confusing base64 with encryption